

CS418 Project 3: Low Latency DASH Streaming

Emirhan Alabaş
Utku Şahin

1 Introduction and Problem Statement

This project implements a low-latency live video streaming platform. The goal is to deliver real time video from a capture device to a browser based player with end-to-end latency below 5 seconds. Low-Latency DASH (LL-DASH) used for chunked CMAF segments and HTTP/1.1 chunked transfer used to allow player to handle segments while they are still being produced. The main requirements for this project include:

- OBS as the media capture and encoding source to produce an RTMP stream.
- A custom FFmpeg build that packages the RTMP input into chunked LL-DASH segments.
- An HTTP serving stack (NGINX plus the modified node-gpac-dash) that delivers chunks to the player as they are produced.
- A dash.js-based player.
- Two experiments measuring how latency changes with seg_duration and frag_duration.

2 Environment and Hardware Setup

The pipeline runs on Ubuntu 24.04 inside WSL2 on Windows 11, with OBS running on Windows. WSL2 is configured in mirrored networking mode so OBS can push RTMP to localhost:1935 on the Windows side, where it reaches FFmpeg listening inside WSL2. FFmpeg, NGINX and node-gpac-dash all run inside WSL2 and the browser player runs on Windows.

- Host OS: Windows 11
- Linux environment: Ubuntu 24.04 (WSL2)
- FFmpeg: built from source
- OBS Studio: x264 encoder, 1280×720 @ 30 fps, baseline profile, 2500kbps CBR
- Web server: NGINX 1.24 on port 8080, with CORS headers and no-cache for live DASH
- Chunk delivery: node-gpac-dash (Node.js 24) on port 8000
- Player: dash.js

3 SDKs and Tools Used

The SDKs and open-source components used in this project are listed below.

Server side:

- **FFmpeg** compiled from source with a one-line patch to libavformat/dashenc.c that removes the tmp rename so node-gpac-dash can read segments while they are still being written.
- **node-gpac-dash** modified Node.js HTTP server that performs HTTP/1.1 chunked transfer encoding, sending each CMAF fragment to the player as soon as FFmpeg writes it.
- **NGINX** serves the MPD manifest, segment initialization

Client side:

- **dash.js** DASH player with low-latency mode.
- **jsQR** JavaScript QR-code decoder used for latency measurement.
- **qrcodejs** used to render a QR code encoding in OBS for the current system time in milliseconds.
- **HTML5, Canvas and Media Source Extensions (MSE)** for video rendering and per-frame QR extraction.

Capture side:

- **OBS Studio** captures the real time video, composites the system-time text overlay and the QR-code onto the scene, encodes with x264, and pushes the result over RTMP to FFmpeg.

4 Pipeline and FFmpeg Capture

The full pipeline is: OBS (Windows) to RTMP to patched FFmpeg to LL-DASH segments to NGINX and node-gpac-dash to dash.js player. FFmpeg listens for the RTMP connection from OBS, passes the H.264 stream and writes LL-DASH output. The command-line flags are listed in Table 1.

Flag	Purpose
<code>-f flv -listen 1 -i rtmp://...</code>	Listen for an inbound RTMP connection from OBS
<code>-c:v copy -an</code>	Pass H.264 video through without re-encoding; drop audio
<code>-ldash 1</code>	Enable LL-DASH output mode
<code>-streaming 1</code>	Write each fragment as soon as it is produced
<code>-use_template 1 -use_timeline 0</code>	Number-template segment naming
<code>-seg_duration <value></code>	Segment duration in seconds
<code>-frag_duration <value></code>	Fragment duration in seconds
<code>-frag_type duration</code>	Fragment by wall-clock duration instead of frames
<code>-utc_timing_url "..."</code>	Adds UTCTiming descriptor to the MPD
<code>-window_size 15</code>	Number of segments kept on disk
<code>-extra_window_size 15</code>	
<code>-remove_at_exit 1</code>	Remove output files on clean shutdown
<code>-f dash /var/www/html/ldash/1.mpd</code>	Write DASH manifest under NGINX's web root

Table 1: FFmpeg LL-DASH command-line flags.

The DASH output files written to `/var/www/html/ldash/` include:

- `1.mpd`: the DASH manifest.
- `init-stream0.m4s`: initialization segment for the video track.
- `chunk-stream0-XXXXX.m4s`: video data segments composed of one or more fragments.

5 Server Configuration

Two HTTP servers run side by side. NGINX serves static assets (player HTML, MPD, init segments) on port 8080 with the following configuration:

- CORS headers so `dash.js` can fetch from a different origin.
- Cache-Control: no-cache on every response so the player always retrieves a fresh MPD.
- Document root `/var/www/html` (where the DASH output and `player.html` live).

The modified `node-gpac-dash` runs on port 8000 with the flags `-chunk-media-segments -cors -chunks-per-segment N` where `N` is set per experiment to match `seg_duration/frag_duration`. As soon as FFmpeg writes a new fragment to disk `node-gpac-dash` pushes it to the player over an open HTTP chunked response.

6 Client Architecture and Custom Player UI

The client is a single HTML page using `dash.js` for playback and `jsQR` for accurate latency measurement. Key features:

- Low-latency `dash.js` settings: `liveDelay=0`, `liveCatchup.enabled=false`, and a small buffer (`bufferTimeAtTopQuality=4`, `bufferToKeep=4`) so the player stays close to the live edge without aggressive catch-up speed changes that would distort latency measurements.
- Millisecond-precision system clock rendered with vanilla JavaScript and updated every 50 ms used for visual comparison with the OBS clock overlay burned into the video.
- QR-code decoder loop running at 10 Hz: each iteration draws the current video frame to a hidden canvas, runs `jsQR` on it, parses the decoded text as a millisecond timestamp, and computes the difference against `Date.now()`.

- Rolling 10-second window of QR samples whose average is displayed continuously. This stabilizes single-frame noise into a clean number for each measurement.
- dash.js live-edge latency from `getCurrentLiveLatency()` displayed separately. This value reports only the offset from the MPD live edge to the playback position and does not include encoder/ingest latency so it is shown with the QR measurement to make the difference explicit.

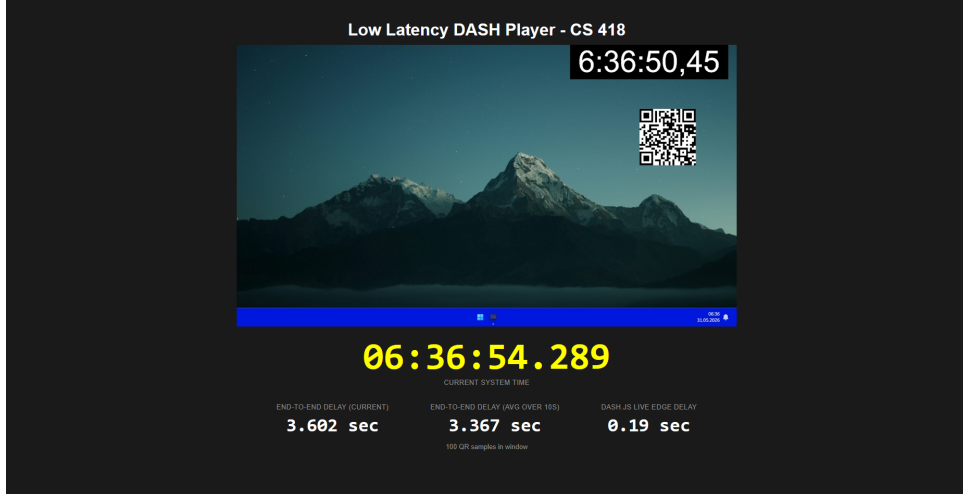


Figure 1: General View of Player.

7 Latency vs. `seg_duration`

In this experiment, the OBS keyframe interval and FFmpeg’s `-seg_duration` are varied in lock-step, while `frag_duration` is held at 1 second. The keyframe interval is set to $30 \times \text{seg_duration}$ frames (at 30 fps) so that every segment boundary aligns with a keyframe. All other parameters (resolution, bitrate, fps, encoder preset, player settings) are kept constant. Results are shown in Table 2 and Figure 2.

keyint	seg_duration (s)	End-to-end latency (s)	dash.js live-edge (s)
30	1	4.85	1.56
60	2	4.45	1.14
90	3	3.93	1.10
120	4	3.95	1.13
150	5	3.94	1.12
180	6	3.83	1.03

Table 2: Measured latency as `seg_duration` is varied with `frag_duration` fixed at 1s.

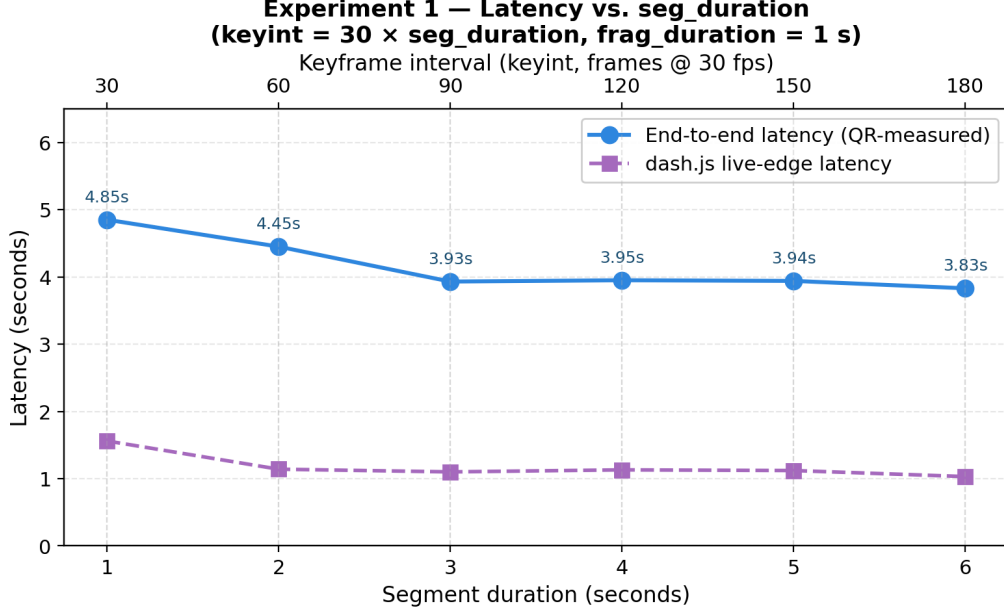


Figure 2: End-to-end latency vs. seg_duration.

End-to-end latency is largely independent of seg_duration once the segment is at least 3 seconds long. Because HTTP/1.1 chunked transfer encoding delivers each fragment to the player as soon as the encoder writes it, the player does not have to wait for full segments to be available. Latency is dominated by frag_duration and the encoder/decoder pipeline, not by the segment boundary. The dash.js live-edge latency is constant nearly at 1.1s across all cases confirming that the segment size is not the bottleneck.

8 Latency vs. frag_duration

Keyframe interval and seg_duration are fixed at 120 and 4s respectively and frag_duration is changed. The node-gpac-dash-chunks-per-segment parameter is updated to match $4/\text{frag_duration}$. Results are shown in Table 3 and Figure 3.

frag_duration (s)	chunks/segment	End-to-end latency (s)	dash.js live-edge (s)
0.033	121	-	-
0.066	61	-	-
0.1	40	3.15	0.23
0.2	20	3.78	0.32
0.5	8	3.91	0.61
1.0	4	4.51	1.11
2.0	2	5.36	2.14
4.0	1	7.81	4.54

Table 3: Measured latency as frag_duration is varied with seg_duration fixed at 4s.

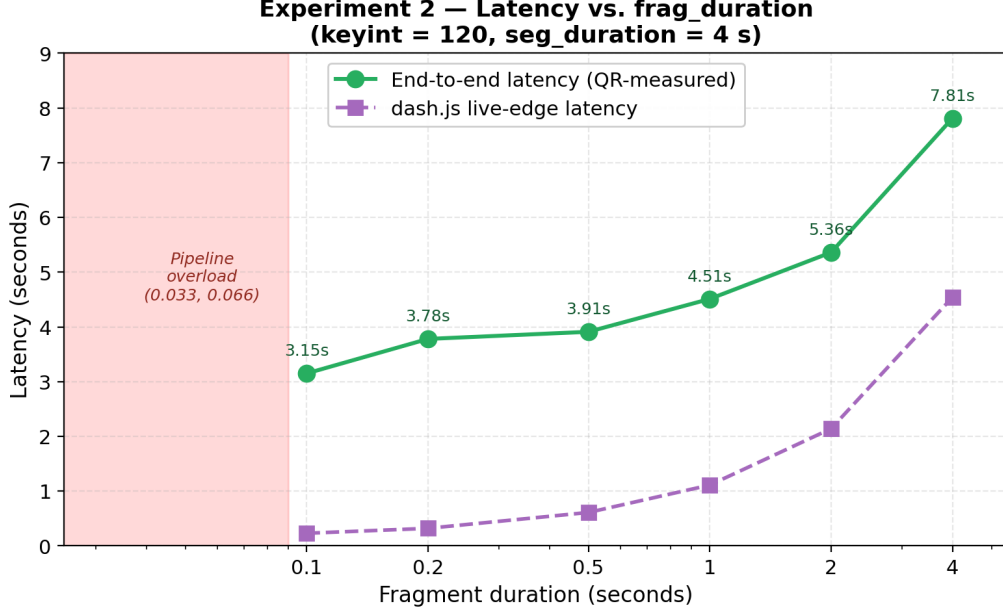


Figure 3: End-to-end latency vs. frag_duration.

Varying frag_duration produces a clear observation which is end-to-end latency rises from 3.15s at frag_duration=0.1s to 7.81s at frag_duration=4s. The frag_duration parameter directly controls how often the encoder emits a new chunk and therefore how often the player receives new video data. Smaller fragments produce a continuous data stream and minimize the time between encoder output and player usage. Also frag_duration at 0.033s and 0.066s node-gpac-dash could not send fragments as fast as FFmpeg generated them so the server fell behind every fragment until the player stalled.

9 Conclusion

We built and validated a low-latency live streaming pipeline using OBS, a patched FFmpeg, NGINX, node-gpac-dash, and dash.js. Under 5 second end-to-end latency was achieved. The experiments confirm LL-DASH architecture. The QR-based measurement method gave much more precise readings than visual clock comparison.